Recursion vs. Iteration: At What Input Size Does The Overhead of Recursion Make an Iterative Approach Significantly More Efficient In Practice?

Team 14: Amanda Thorpe, Lauren (Kate) Tucker, Jude Wilson, Daniel Vo

INTRODUCTION & MOTIVATION

In Java, recursion is often simple & readable in comparison with iteration, but recursion's repeated opening of stack frames can lead to lengthy wait times or overflows. It is commonly agreed upon that recursion is most viable when input sizes are kept small. To truly test recursion and iteration against each other, we wrote a few different algorithms, each of them recursively *and* iteratively, and checked to see when recursion became inefficient or failed altogether when compared to iteration.

LITERATURE REVIEW

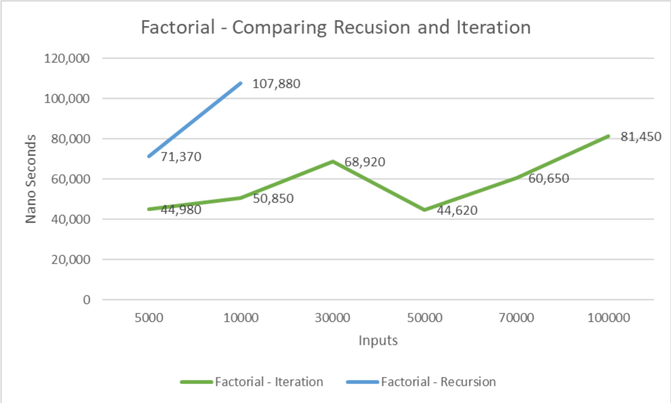
- The MIT *Recursion and Iteration* article explains that both techniques produce patterned and step-by-step evaluation, while recursion does this by repeatedly calling the same function and making a chain of stack frames, contrarily to iteration that uses loops that recycle a single frame, naturally avoiding accumulating overhead.
- This distinction is especially evident as the depth of recursion increases, as each additional call must store parameters, local variables, and return information on the JVM call stack; creates a structure often considered intuitive for list, tree, or graph-like problems, but can become problematic if the input is too large (Massachusetts Institute of Technology, 2022).
- The research study detailed in the article: *We Measured Recursion vs. Iteration in Java: The Results Surprised Us.* by an experienced Full-Stack Developer demonstrates these ideas through controlled experiments; it notes that while recursion can perform as well as iteration with a lower and more predictable depth, e.g. in Fibonacci and Factorial algorithms, the performance varies as the input sizes or number of executions increase.
- This performance shift is not simply due to recursion being "slow," but rather because the JVM must follow the structure it is given.
- Concludes that recursion becomes less practical when depth is dependent on factors like external input and when call chains rapidly increase, while iteration has strengths in predictable control of memory and stack behavior (Kavya, 2026).
- Similar to our own findings in this research, both sources reveal that determining the most effective and relevant choice between recursion and iteration should not be based on personal preference, but rather on input sizes, complexity of the algorithm, and practical constrains of the JVM (Kavya, 2026; Massachusetts Institute of Technology, 2022).

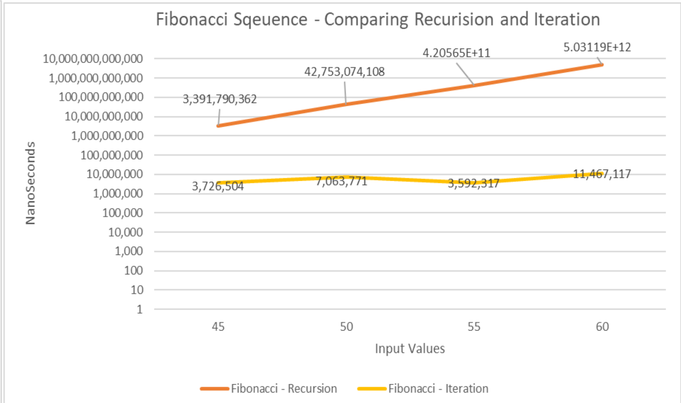
METHODOLOGY

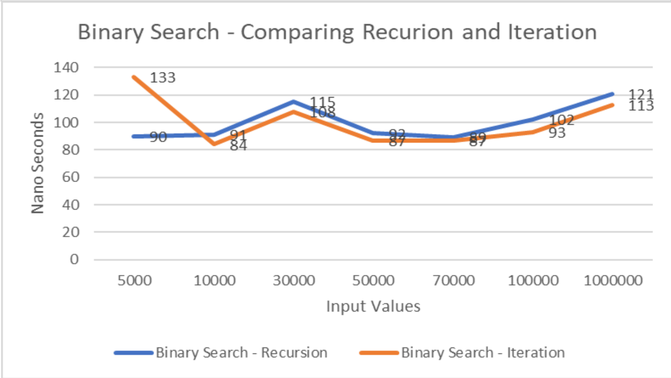
In this project, we implemented four programs, each with both recursive and iterative versions. Fibonacci sequence, factorial, binary search, and fast exponentiation. Each program was designed to ensure a fair comparison between recursive and iterative. The experiments were carried out using the input sizes of 5,000, 10,000, 30,000, 50,000, 70,000, and 100,000, along with a much larger size for binary search. Furthermore, the Fibonacci sequence was also given supplementary, smaller values to test, due to its trouble in testing such large values. When executed, time was measured using System.nanoTime(). Each program was repeated 10 times to calculate the average runtime, reducing random fluctuations and improving accuracy. The program also includes exception handling to detect StackOverflowError, allowing identification of the recursion cliff for each recursive implementation.

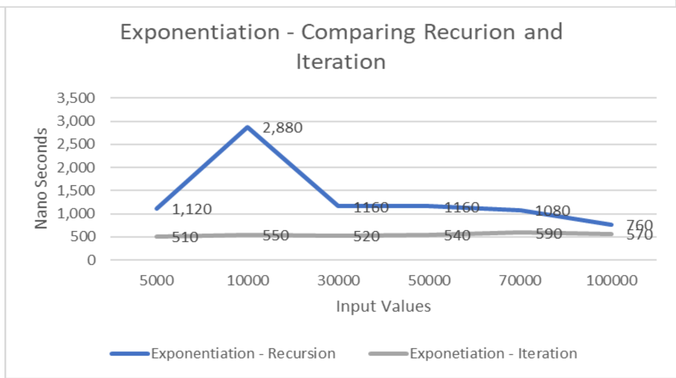
RESULTS OF DATA COLLECTED

- Fibonacci – When testing the original data set of 5k to 100k, none of the input values would ever finish, nor would they give a StackOverflow. Thus, extra smaller values were tested to build data and a chart; these input values yielded results where recursion got exponentially longer to finish, while iteration was always quick.
- Factorial – Recursion managed to finish the first two inputs of 5k and 10k, though at a slower pace than iteration. With all larger input values, recursion only ever gave a StackOverflow error, and never achieved an answer.
- Binary Search – Both iteratively and recursively, binary search was able to complete the input values very quickly. Recursion always stayed comparable to iteration in speed.
- Fast Exponentiation – Recursion was always slower than iteration, though it managed to get all input values done fairly quickly. Interestingly, recursion had an outlier on 10k, where it was noticeably at its slowest.









KEY CONCLUSIONS

- Iteration proved to be a more dependable method in cases of large input sizes and when computation depth cannot be easily controlled or predicted.
- While recursion may be favored for problems that require being sectioned into smaller components, the results show that its performance significantly declines when repeated calls accumulate on the JVM call stack, i.e. in the Factorial and Fibonacci tests where recursion slowed significantly or failed to compile altogether with a StackOverflowError.
- Contrarily, iteration maintained steady runtimes in all four algorithms that were tested, even completing the largest input values without stack overflows or slow performance.
- Exceptionally, the binary search algorithm proved that recursion was a successful and reliable method in cases where the depth is limited in some way or inside of the algorithm itself.
- Ultimately, developers should prioritize recursion in cases where the problem requires a structure with clear, established boundaries, but should favor iteration when input sizes are large and when memory usage must be controlled. A developer can determine whether to use recursion vs. iteration based on the input size, structure and depth of the algorithm, and JVM behavior for repeated calls during execution.

REFERENCES

Kavya. (2026, February 14). We Measured Recursion vs Iteration in Java. The Results Surprised Us. Medium. https://medium.com/@kotiavula6/we-measured-recursion-vs-iteration-in-java-the-results-surprised-us-81dce2166c65

Massachusetts Institute of Technology. (2022). Recursion and Iteration. Web MIT. https://web.mit.edu/6.102/www/sp23/classes/11-recursive-data-types/recursion-and-iteration-review.html

ACKNOWLEDGEMENTS

Many thanks to Dr. Mudduluru and research assistant Vishnu Kadiyala for providing guidelines, assistance, and help with the project and its requirements.

RESEARCH POSTER PRESENTATION DESIGN © 2011
www.PosterPresentations.com